



universität
wien

BACHELORARBEIT / BACHELOR'S THESIS

Titel der Bachelorarbeit / Title of the Bachelor's Thesis

„Polyglot Files in Semantic Data: Detection and Sanitization“

verfasst von / submitted by

Luca Schrott

angestrebter akademischer Grad / in partial fulfilment of the requirements for the degree of

Bachelor of Science (BSc)

Wien, 2026 / Vienna, 2026

Studienkennzahl lt. Studienblatt /
degree programme code as it appears on
the student record sheet:

UA A 033 521

Studienrichtung lt. Studienblatt /
degree programme as it appears on
the student record sheet:

Bachelorstudium Informatik

Betreut von / Supervisor:

Univ.-Prof. Dipl.-Ing. Mag. Dr.techn.Edgar Weippl

Abstract

This thesis addresses the problem of polyglot files. Polyglot files are increasingly used by malicious actors to fool defensive tools and avoid file type restrictions. Consequently, we investigated the capabilities of current polyglot file detection and sanitization tools. Six distinct polyglot generators were developed: two targeting non-essential structural regions, and four embedding payloads within semantically essential data, such as image content and valid PDF text streams. The techniques cover the BMP, PNG, JPEG, and PDF file formats. We then evaluated four different file type detection tools on samples generated using the created generators as well as existing polyglot generation techniques. These include the well-known file tool, the deep learning-based Magika, and two tools specialized in polyglot detection. The aim of this evaluation was to empirically compare the effectiveness of the presented techniques with existing ones and to assess Magika's polyglot detection capabilities. We also tested the employed techniques against three Content Disarmament and Reconstruction (CDR) tools. We found that no detection tool had a recall higher than 65% and that the specialized detection tools are susceptible to false positives on image data. Regarding the CDR tools, we found that they performed well at removing non-essential structures in the tested files. However, every tool failed to sanitize at least one of the techniques presented here.

Thus, we concluded that using automated detection tools may pose problems due to lacking recall as well as a large number of false positives when considering the ratio of benign to polyglot files in the real world. While a more aggressive and comprehensive sanitization might remove the presented and similar techniques, we also deem it important for future research to consider the potential usability and integrity cost of comprehensively sanitizing semantic contents of a file.

Contents

Abstract	i
1. Introduction	1
1.1. The danger of polyglot files	1
1.2. Evaluation of Magika	1
1.3. Hiding payloads in semantically relevant regions	2
2. Background	3
2.1. BMP	3
2.2. PNG	4
2.3. JPEG	4
2.4. PDF	6
2.5. Taxonomy	6
3. Related Work	8
3.1. Language-Theoretic Security	8
3.1.1. Parser Differentials in Practice	8
3.2. Polyglot Files	8
3.2.1. Polyglot File Techniques	9
3.2.2. Detection of Polyglot Files	9
3.2.3. Sanitization and Reconstruction	9
3.2.4. Near-Polyglots and Crypto-Polyglots	9
4. Polyglot File Generation Techniques	10
4.1. BMP Pixel Data Injection	10
4.2. PNG Pixel Data Injection	11
4.3. PNG ICC Injection	11
4.4. JPEG APP0 Injection	13
4.5. JPEG Data Injection	13
4.6. PDF Invisible Text Injection	14
4.7. Taxonomy of Presented Techniques	16
5. Implementation	18
5.1. Generation, detection and evaluation framework	18
5.1.1. Detector tools	18
6. Evaluation	20
6.1. Testing of polyglot file functionality	20

6.2. Evaluation on detection tools	20
6.2.1. False positive rate	20
6.2.2. Polyglot file detection	20
6.2.3. Reproducibility	23
6.3. Evaluation on CDR tools	23
6.3.1. Tested tools	23
6.3.2. Test methodology	24
6.3.3. Results	24
 7. Discussion	 26
7.1. Detection of polyglot files	26
7.2. Content Disarmament and Reconstruction	26
7.3. Future Work	27
7.3.1. On detection	27
7.3.2. On Content Disarmament and Reconstruction	27
 8. Conclusion	 29
 Bibliography	 30
 A. Appendix	 35
A.1. Usage of AI tools	35

1. Introduction

What is a polyglot file? A polyglot file is a file that is valid as at least two distinct file formats. In practice, this implies that the relevant parsers for both file formats will accept the polyglot file.

The problem introduced by polyglot files: A file might indicate its format via file extension, shebang, or magic bytes. While this can be easily verified by parsing the file as the indicated format, ensuring that this is the *only* format the file is valid as presents a much more challenging task. Consequently, polyglot files can be utilized to smuggle a covert file of a type that would otherwise raise suspicion or be filtered out. For example, during an image upload, a system might validate that the file is indeed a legitimate image. However, this validation does not exclude the possibility of the file also being valid as a malicious script or archive.

1.1. The danger of polyglot files

Polyglot files represent an emerging threat to cyber infrastructure. Koch et al. [KOS⁺25] conducted a survey of polyglot files found in the wild, as well as recent attack chains, finding that many advanced attackers abuse polyglot files to deliver malware. These files can bypass security boundaries easily, becoming apparent only when their hidden contents are extracted or executed. In contrast to steganographic payloads, the covert file in a polyglot can be executed or extracted using standard tools. The authors further noted the significance of script languages such as HTA, JavaScript, PHAR, and PHP in the real-world polyglot threat landscape. These script languages are highly versatile and robust; their parsers often ignore invalid inputs or continue execution despite errors. Thus, they are especially prevalent among malicious polyglot samples [KOS⁺25]. This motivated the exploration and evaluation of the existing defense landscape.

1.2. Evaluation of Magika

Magika [FIF⁺25] is a neural network developed by Google Research to identify file formats. It is computationally lightweight and highly efficient. It is particularly relevant to security research, having been integrated into VirusTotal and Gmail, among others. However, to date, there has been no evaluation of its capability to detect polyglots [FIF⁺25]. Koch et al. [KOS⁺25] evaluated multiple file type identification tools, as well as specialized polyglot detection programs. They subsequently developed a neural network-based polyglot detector, Polyconv, which has not been publicly released. They noted that they did not evaluate Magika because it outputs only a single label [KOS⁺25]. Although

1. Introduction

Magika’s public interface provides a single label, it is possible to access the probability distribution of the model’s output. This motivated us to evaluate Magika’s utility for polyglot file detection.

1.3. Hiding payloads in semantically relevant regions

Polyglot files found in the wild are typically simple stacks or parasites [KOS⁺25] (see 2.1), which mostly embed data into comments, metadata fields, or regions outside the file boundary. There exists related research [BSS23] on identifying regions in files ignored by parsers. Similarly, research into neural networks using structural features to detect malicious JPEG files with embedded scripts [CNE20] has shown promising results, implying that these files fundamentally alter the file structure in ways not seen in benign files. Existing tools for creating polyglot files, such as Mitra [Alb20] or Fazah [KOS⁺25], have focused mainly on creating stack or parasite polyglots. Furthermore, Content Disarm and Reconstruction (CDR) approaches, such as IMSan by [KOS⁺25], intend to strip away all but the most vital data needed for the file. Meanwhile, solutions that target the essential data of the file, like [JLE20], [BDD23], and [Dub23], are focused primarily on mitigating exploits or steganographic payloads. These solutions are diligent and comprehensive but are not widely deployed and can negatively impact file usability and fidelity.

This inspired the present research to embed payloads in semantically vital regions of the host format to evaluate the impact on both detection and sanitization. It was found that documentation on some of the techniques already exists [Mor25a], but they have not been thoroughly evaluated. We developed a generation tool for each of our techniques and evaluated them against standard file type detection tools, such as the `file` command and Magika (mentioned above), as well as specialized tools for polyglot detection: PolyFile [Tra19] and PolyDet [CL18]. We also evaluated the techniques against CDR tools, including a reconstructed version of IMSan, which successfully sanitized 100% of all polyglots generated using the Fazah tool [KOS⁺25], and the commercial CDR services Glasswall Halo and OPSWAT MetaDefender.

2. Background

As BMP, PNG, JPEG, and PDF are used as host formats, this section provides background on the internal functioning of these file formats. The reader should note that this is a concise description and does not cover all features of the formats, but rather focuses on the fundamental components and the information required to understand the polyglot techniques.

2.1. BMP

BMP is a relatively simple format as it is fundamentally only composed of two headers as well as the pixel data in its raw form [Mica]. The first "fileheader" begins with the magic sequence "BM" enforced at offset 0, followed by the file size and the offset to the start of the pixel data [Micb]. Then follows the DIB-Header. The only common DIB-Header used on modern systems is the bitmapinfoheader [Micc]. Multiple bitmapinfoheader versions exist (e.g., [Mice], [Micd]), and each version is a superset of the previous one, adding new fields. The header defines width, height, bits per pixel (thus defining modes such as grayscale, RGB, RGBA) and metadata for the (optional) color palette. It also defines a field to indicate the compression method. A color palette may follow after the DIB-Header. Then the pixel data begins at the offset previously defined in the base header (so there may be a gap). The pixel data is composed of pixels with the size of each pixel defined by the bits per pixel field. It is defined row by row with each row padded to a multiple of 32 bit. A rough overview of the format structure can be seen in Figure 2.1

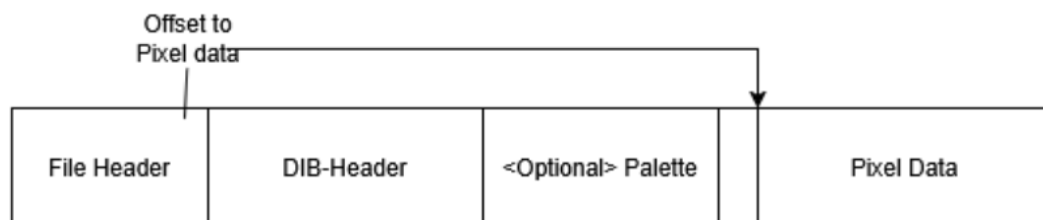


Figure 2.1.: BMP File Format Structure

2. Background

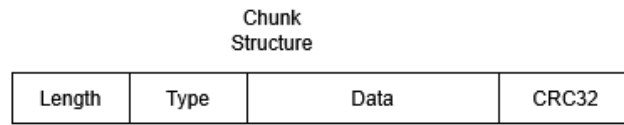


Figure 2.2.: Structure of a PNG Chunk

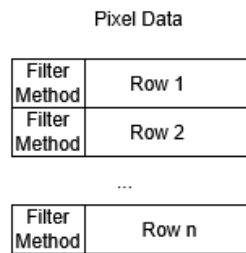


Figure 2.3.: PNG pixel data structure before compression

2.2. PNG

The PNG file format [PNG03] is fundamentally composed of the magic byte sequence followed by a number of chunks. Each chunk is a struct containing length, type (4-byte ASCII), its data (if any), and finally a CRC over the type and data as visualized in Figure 2.2. If the first letter of the type is uppercase the chunk is one of the critical chunks which every parser has to implement. Any other chunks (ancillary) must start with a lowercase letter and can be ignored by parsers. Among the critical chunks are IHDR, which is mandatory and must be the first chunk. It defines width, height, color type, bit depth ("bits per color"), and whether the pixel data is interlaced. The next important critical chunk is the IDAT chunk containing the pixel data. The raw pixel data is specified row by row (if not interlaced), then a filtering algorithm is applied to each row to improve compressibility. Subsequently, the row is prepended with a byte to specify the filtering method used (0 being none) (See Figure 2.3) and then wrapped in the ZLIB format [DG96] using deflate compression [Deu96]. The entire compression process is always lossless. The standard also defines an optional PLTE palette chunk. The empty IEND chunk marks the end of the PNG. The general structure of a basic PNG file is visualized in Figure 2.4

2.3. JPEG

In the context of file formats, references to JPEG typically denote the JPEG File Interchange Format (JFIF) [Ham92]. JPEG itself is the standard for coding and compressing image data [ITU92]. The JPEG File Format consists of multiple segments where the start of a segment is signified by a 2 byte marker with its first byte being `0xFF`. Markers

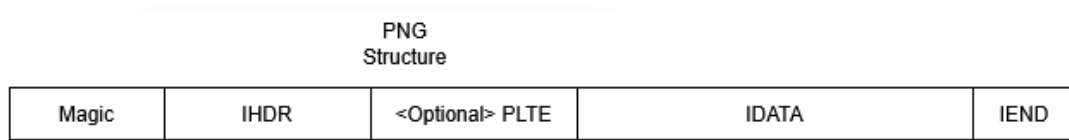


Figure 2.4.: Structure of a PNG File

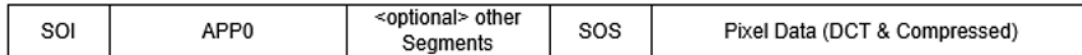


Figure 2.5.: JPEG File Structure

indicating additional data are followed by a 2-byte length field.

The file is wrapped between standalone SOI (StartOfImage) and EOI (EndOfImage). SOI thus functions as a magic byte for the file format. The other tags which are in every JPG are those needed for the compression such as defining Huffman (DHT) or quantization tables (DQT). The SOS Tag specifies the start of the entropy coded data. The JFIF standard also defines multiple metadata segments, where the APP0 segment is mandatory right after the SOI segment. The APP0 segment (see Figure 2.6) can contain a thumbnail, though modern images or display tools do not seem to use this feature. One can also specify other optional APP segments, such as the APP1 segment containing EXIF metadata [CIP19]. APP2 is often used for embedding ICC profiles [ICC22]. There also exist APP3-APP11 which don't have a standard use case. The JPEG standard also defines a COM comment segment which allows arbitrary text that gets displayed by some image viewers [Ham92]. An overview of the file structure can be seen in Figure 2.5.

The compression algorithm acts on blocks of 64 pixels each. Fundamentally the compression consists of first performing a Discrete Cosine Transform on the block, lossy quantization of the coefficients and then lossless compression of the remaining DCT coefficients using Huffman coding. Fundamentally, two types of JPEG exist, the standard JPEG, where all the DCT coefficients for a section are given at once, and the progressive JPEG, which contains multiple SOS segments. This allows the DCT coefficients for each block to get refined further every segment [ITU92], thus allowing smoother rendering for potentially slow environments like a webpage.

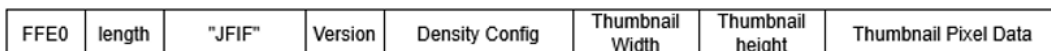


Figure 2.6.: APP0 segment Structure

2. Background

2.4. PDF

PDF is arguably the most complicated of these formats. A more comprehensive description of this file format can be found in Rimser [Rim24]. A PDF consists of a body, xref table and trailer and is parsed bottom up starting from the trailer. As the xref table and trailer are not relevant to the technique presented in this thesis they are omitted from the description.

Object structure. The body of a PDF is composed of numbered objects. Each object has an object number and a generation number, which is almost always 0 thus for simplicity assumed 0. So they are declared by `objnum 0 obj ... endobj` where `objnum` is replaced by the identification number of the object. Objects can be referenced by `objnum 0 R`. Objects can contain various basic data types, including dictionaries, streams, arrays, and names [ISO08].

Data types. Names start with a `/` like `/Name`. Arrays consist of multiple elements inside of squared brackets `[e1 e2 e3]`. Streams are between `stream` and `endstream` keywords and can contain data like texts or images. Streams can employ `zlib` compression to reduce the PDF size. Dictionaries consist of key value pairs like `<< Key Value Key Value... >>` [ISO08].

Object types. An object type is mainly defined by its dictionary with a `/Type` key and a corresponding value. Common Types include `/Pages`, which references all page objects for the PDF and is thus central. `/Page` defines an individual page. The individual page contents are defined in the `/Contents` dictionary by stream elements which are in another, referenced, object. Fonts (`/Font` dictionary) or Images (`/XObject` dictionary) used for the stream are contained in the `/Resources` Dictionary [ISO08].

Streams. The stream has a dictionary defining arguments such as compression and, if necessary, its type. Furthermore a stream can employ fonts which are declared in the corresponding page object containing the stream, either inline or as reference to a Font Object [ISO08].

A visualization of a sample pdf file structure can be found in Figure 2.7.

2.5. Taxonomy

Ange Albertini proposed a taxonomy, as represented in [KOS⁺25] and [ADG⁺22] (see Table 2.1).

Stack Polyglots are polyglots where one file format is simply appended to the other; an example would be appending a RAR file after the EOI marker in a JPEG. Parasite polyglots hide in comments or metadata, i.e., non-essential but valid parts of the document. An example of comment would be putting data into one of the JPEG App segments (see 4.4). Zippers are polyglots where both embedded files employ parasitic embedding on each other. So far, only DICOM has been found to be compatible (see [Alb15]). Cavities are padding space. These are also rare and found mostly in files that begin at some offset that is not 0.

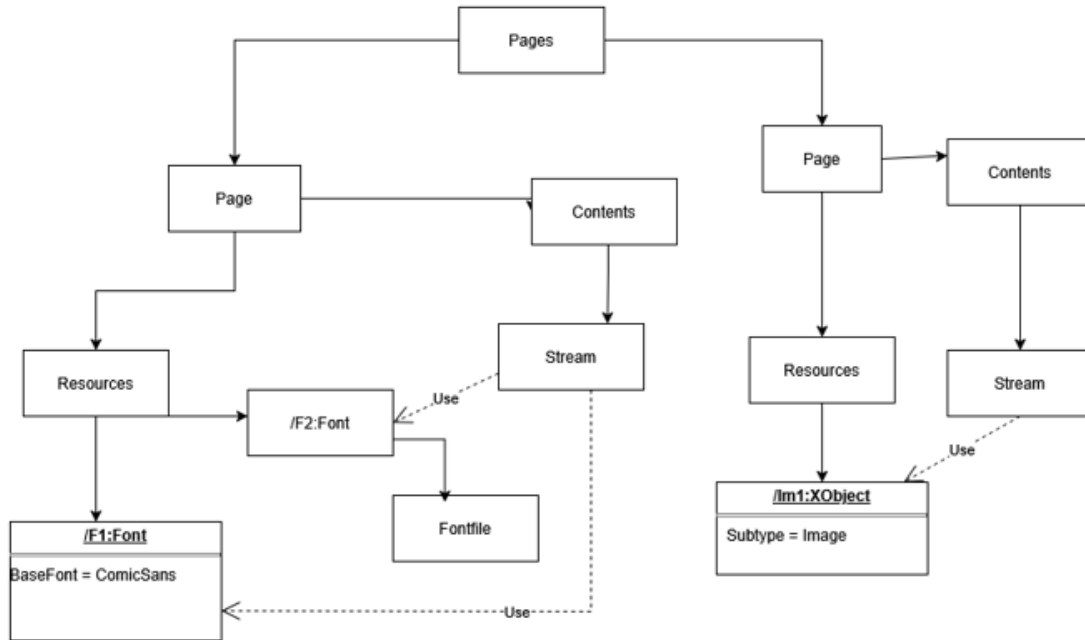


Figure 2.7.: PDF example body structure

Type	Embedding Region
Stack	After EOF
Parasite	Comments/metadata
Zipper	Mutual comments
Cavity	Padding space

Table 2.1.: Polyglot taxonomy

3. Related Work

3.1. Language-Theoretic Security

Language-Theoretic Security (LangSec) uses formal language theory to identify the underlying reasons that make modern-day computer bugs and exploits possible. One class of bugs central to LangSec research is parser differentials, which occur when two parsers responsible for parsing the same input language disagree on an input [AS23]. A key observation is that parsers can have different levels of complexity. One principle that LangSec advocates is using the least complex parser able to parse the input language, matching the parser’s complexity to the input language [SPB12]. Research has shown that for simple parsers (up to Context-Free Grammars), it is possible to test for parser equivalence. However, this becomes undecidable for more complex parsers such as Turing Machines [Ali25, Ana22]. It follows that restricting file format complexity is necessary to verify parser equivalence. Such principles led to the creation of PDF/Restricted, which constrains PDF complexity to a subset of Context-Free Grammar [ELM16]. LangSec also strongly advocates against shotgun parsing, where input parsing and validation are mixed with processing logic and scattered throughout the program. This approach has been shown to lead to various bugs; the recommended alternative is to first parse the entire input, then process it [Ana22].

3.1.1. Parser Differentials in Practice

Parser differentials occur in many formats. The most academically explored case is schizophrenic PDF files, which display different contents depending on the viewer used, as most PDF viewers are error-tolerant and do not fail when files deviate from specifications [Rim24]. Robinson [Rob20] uses multiple PDF parsers to detect specification non-compliance. Several papers address parser differentials in barcode scanning: Dabrowski et al. [DKUW14] demonstrated hiding different barcodes inside a host QR code, while Chou et al. [CW24] showed QR codes that parse differently depending on scanning location. Klein et al. [KJ24] bypassed HTML sanitizers by exploiting parsing differentials between sanitizers and browsers.

3.2. Polyglot Files

Polyglot files are distinct from the parser differentials described above. Parser differentials arise from two parsers of the same format disagreeing on a file. Polyglot files arise not from parser disagreement but from format specifications not being mutually exclusive, or from parsers being overly permissive in accepted inputs.

3.2.1. Polyglot File Techniques

Early work on polyglot files dates back to Albertini, who in the journal PoC||GTFO [Alb15] documented many polyglot techniques and developed the foundational taxonomy.

Mitra [Alb20] provides a generation framework for polyglots, allowing creation of various polyglot types with known formats as well as injecting arbitrary data into host formats. Its techniques vary by format but generally employ all methods from the taxonomy. Glasswall’s white paper [Mor25a] documents many polyglot techniques, including the BMP pixel injection technique employed in this thesis as well as the concept of embedding non-visible text in valid PDF streams.

3.2.2. Detection of Polyglot Files

Koch et al. [KOS⁺25, KOC⁺22] developed neural networks for polyglot file detection. Their PolyConv convolutional neural network [KOS⁺25] achieved strong detection results but was not publicly released. Other researchers [CNE20] built a neural network using only ten structural features to detect embedded scripts in JPEG files with high accuracy. Brodin et al. [BSS23] explored blind spots in file formats using PDF as an example, identifying regions ignored by parsers that do not affect output. Such detection could help identify covert content in these regions, though it may not address payloads in semantically essential data.

3.2.3. Sanitization and Reconstruction

Most academic work on sanitization and reconstruction has not primarily focused on polyglot files. Koch et al. [KOS⁺25] created ImSan for reconstructing image files from scratch to remove covert content; we evaluate a reconstruction of this tool against the techniques presented in this thesis. Most other work targets exploits and steganographic embeddings. Dubin [Dub23] created a comprehensive PDF sanitization tool with effective threat removal but at the cost of decreased usability and invalidated digital signatures. Glasswall released a white paper on image steganography defenses [Mor25b]. Jung et al. [JLE20] and Belkind et al. [BDD23] created CDR tools for image sanitization, applying various transformations to neutralize steganographic threats while preserving image quality.

3.2.4. Near-Polyglots and Crypto-Polyglots

Albertini et al. [ADG⁺22] define near-polyglots as files valid in only one format but differing by a small overlapping region (such as magic bytes) from a file of another format. Changing this region switches the file type. Crypto-polyglots are near-polyglots where this region can be switched by a cryptographic operation. The paper demonstrates ciphertexts that, under keystream-based ciphers, decrypt as one format under key K_1 and another format under key K_2 , exploiting the lack of key commitment in some common authenticated encryption schemes.

4. Polyglot File Generation Techniques

Note: In the diagrams visualizing the techniques, the red color represents a data chunk containing the payload. Green represents a change or addition necessary for the technique to work but which does not directly contain any payload data.

4.1. BMP Pixel Data Injection

Since BMP pixel data consists of bytes representing colors with no strict rules regarding the values of either the pixels or the padding, it presents a prime target for overwriting with arbitrary bytes. To achieve this, the pixel data is directly overwritten, as visualized in Figure 4.1.

To implement this technique, the header is first parsed to locate the image offset and calculate the image size: `imgSize = width * length * bytespp`. Subsequently, the payload overwrites the data shortly after the beginning of the pixel array:
`bmp_array[offset:offset_end] = payload.`

It is recommended to embed the payload in either the first or last rows of the image, as data corruption is typically less visible in these regions. The primary advantage of this technique is the absence of technical constraints beyond the image size. The significant disadvantage, however, is that if the payload is large relative to the image size, the pixels defined by the payload data will be visible upon close inspection. This technique has been previously covered by [Alb15] and [Mor25a].

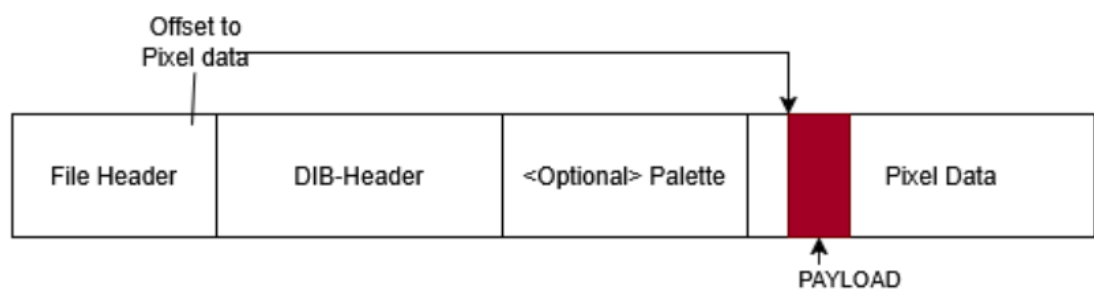


Figure 4.1.: BMP Injection into Pixel Data

4.2. PNG Pixel Data Injection

Injecting payload data into PNG pixels is more challenging because PNG employs both filtering and compression. Most filter options make subsequent bytes or rows dependent on previous ones; consequently, altering bytes in an upper row risks corrupting subsequent data. Therefore, it is advisable to set the filtering method to 0 (No filtering). Additionally, the IDAT pixel data must be wrapped in zlib compression using the deflate method. This requires a 2-byte zlib header and an Adler-32 checksum at the end of the data. Although the zlib header contains a `compress_level` field, its definition is purely indicative and is not utilized by decoders [DG96].

Deflate compression [Deu96], the only compression method currently defined for zlib, compresses the filtered pixel data into a series of blocks. Each block is prepended by 3 header bits. The first bit, BFINAL, indicates if it is the last block. The following 2 bits, BTYPE, indicate the block type. Type 00 indicates to the decoder that no compression was used. Combining a BTYPE of 0 with a Filtering Method of 0 makes it possible to embed literal pixel data into the PNG. Consequently, one can overwrite the data with the covert file's raw data, similar to the BMP method.

For implementation, pixels must first be extracted row by row from the PNG. This was achieved using the `pypng` library¹. A portion of the very first row can then be overwritten with the payload data. It is essential to prepend every row with a 0 byte, as shown in Figure 4.2, to indicate an unfiltered row. The Python `zlib` library can then conveniently handle the ZLIB and DEFLATE requirements by instructing it to avoid compression: `compressed = zlib.compress(bdata, level=zlib.Z_NO_COMPRESSION)`. To ensure correctness, the PNG was rebuilt minimally from scratch:

```
full_png = magic + ihdr_chunk + idat_chunk + iend_chunk.
```

A limitation of this technique is that every row must begin with a 0 byte. If this condition is not met, the payload size is limited to `BytesPerPixel * ImageWidth` bytes. Furthermore, the block size for non-compressed blocks is limited by specification to 65,536 bytes. The injection is visualized in Figure 4.2.

4.3. PNG ICC Injection

[Alb20] developed a technique to embed data into an ICC 'junk' chunk. This approach was adapted to inject such polyglot ICC data inside a PNG file. The rationale is that ICC profiles are legitimate and widely used ancillary PNG chunks; therefore, detecting or stripping structural anomalies requires parsing the internal ICC data. A similar technique may be viable for many file formats that support embedded ICC files.

The ICC data format consists of a tag table containing the name, offset, and length of each chunk, followed by the data for various chunks sequentially as specified in the table (see Figure 4.3).

A polyglot ICC profile can be created by adding a table entry named 'junk', a valid designation for arbitrary data [ICC22]. The PNG format also allows embedding ICC files

¹see <https://pypi.org/project/pypng/>

4. Polyglot File Generation Techniques

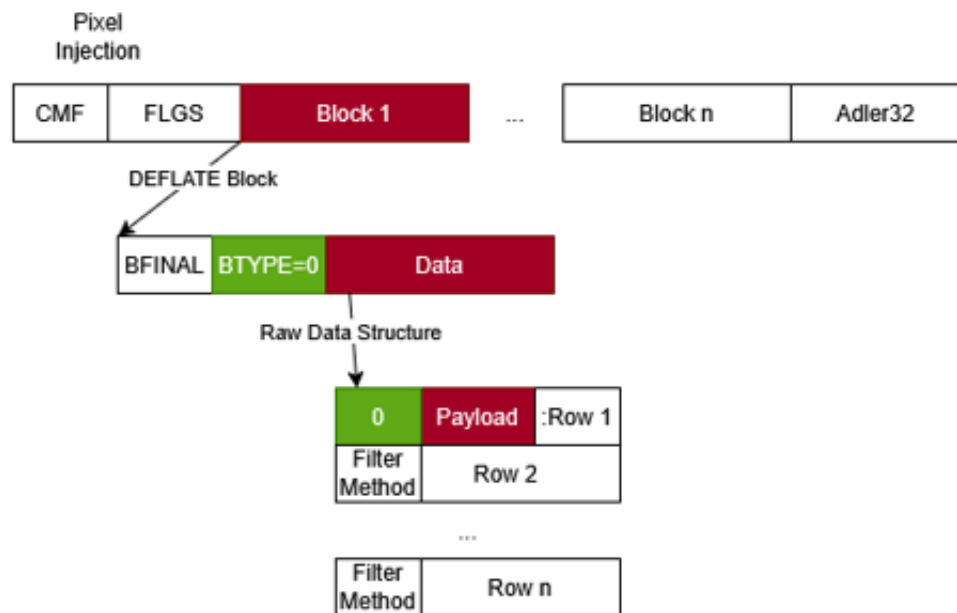


Figure 4.2.: Visualization of the PNG Pixel Injection

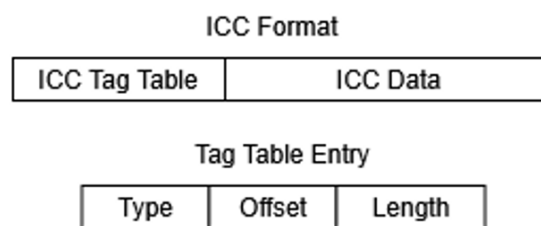


Figure 4.3.: Structure of ICC Data

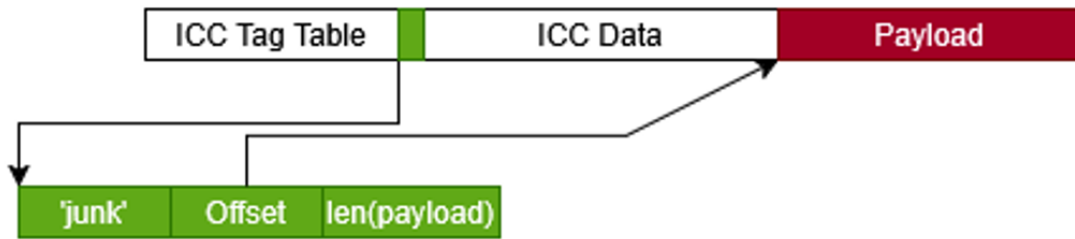


Figure 4.4.: Injecting Payload into ICC Data

in an ancillary iCCP chunk. Thus, it is possible to create an iCCP chunk containing a polyglot ICC file. Because iCCP chunk data must be zlib-wrapped, the same technique (using non-compressed DEFLATE blocks) described in Section 4.2 is employed. The iCCP chunk is then inserted immediately after the IHDR chunk.

To create the polyglot ICC, a valid base sRGB ICC profile was used. First, a new 'junk' entry is added to the ICC Tag Table, increasing the Tag Table size by 12 bytes. Consequently, the offset for every entry in the tag table must be incremented by 12 bytes. The payload data can then be appended to the end of the ICC data. The injection is visualized in Figure 4.4.

A limitation of this technique is that after 65,536 bytes, the DEFLATE block must end and a new block must begin, which would interrupt the payload data. Therefore, using a large ICC profile (close to 65 KB) as a base for the injection leaves little room for a covert file.

4.4. JPEG APP0 Injection

Although the APP0 chunk is mandatory by specification immediately following the SOI marker, its thumbnail data is rarely utilized. Consequently, injecting data into it increases file size but does not fundamentally alter the file structure.

To inject into APP0, one simply appends the payload after the APP0 image data and updates the width and height to accommodate the payload. The data is stored in RGB format, with each pixel containing 3 bytes, allowing it to hold arbitrary data.

Due to the 2-byte size field, the maximum size of the chunk is limited to 65,536 bytes.

4.5. JPEG Data Injection

Injecting into JPEG data cannot be done via literal pixel values because JPEGs are stored using DCT coefficients. Furthermore, there appears to be no method to disable Huffman compression for the coefficients. Thus, it is necessary to directly overwrite the compressed pixel data. To achieve this without causing visual distortion, a progressive JPEG is used. By overwriting data in the last refinement SOS (Start of Scan), only the least significant

4. Polyglot File Generation Techniques

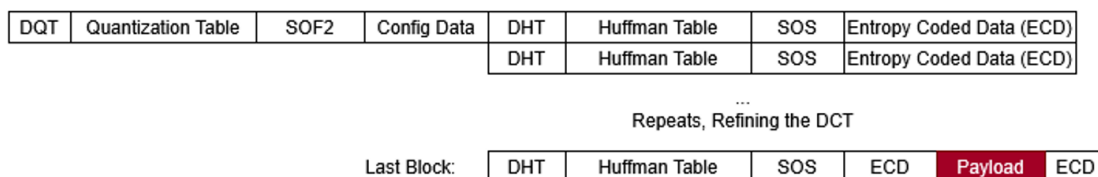


Figure 4.5.: Injecting Payload into progressive JPEG

part of the DCT in those blocks is affected, avoiding the creation of "junk" pixels as seen in 4.1 or 4.2. This technique is visualized in Figure 4.5.

The primary issue is that the coded data is overwritten with arbitrary data without adjusting the Huffman tree. This likely results in the new encoded data decoding to a different length, as the Huffman table was optimized for the replaced data distribution. Although this technically corrupts the JPEG, the resulting polyglot file is accepted by all tested image viewers without warnings. Stricter tools, such as `djpeg` and `jpegtran`, refused to handle the file because the last segment ended prematurely. ImageMagick logged a warning, while Pillow processed the file without issues. It was also observed that repairing the file structure (using Glasswall Halo) involved removing bytes from the end of the final scan data.

A further issue is that in JPEG data, every byte `0xFF` representing data must be escaped by a `0x00` byte, as `0xFF` is reserved for markers. Therefore, the technique is not compatible with covert binary formats. This limitation does not affect covert script files.

Another limitation is that the available space averages approximately 4 KB. However, since this method is primarily used for script languages, this size constraint is generally acceptable.

4.6. PDF Invisible Text Injection

This technique leverages the PDF text content stream feature that allows text to be rendered invisibly by setting the render mode to 3. The technique involves adding a valid PDF content stream where the covert file is embedded as invisible text. The premise is that script code embedded within a valid content stream is not inherently suspicious and has legitimate use cases, yet it will execute successfully when the file is processed by the corresponding interpreter. Furthermore, this technique does not introduce any non-standard structures that might be flagged.

A similar technique was documented by [Mor25a].

First, a new stream object containing a valid font must be created. Inside the stream object, a valid font defined for that page is used, the text rendering mode is set to invisible, and the payload is inserted as the text to display. A page in the document is then located, and the new stream is added to its contents array.

To define a valid font, multiple approaches exist. One could, for example, utilize a font already defined for that page. In this implementation, a new font was added to ensure

better compatibility. A visualization of the technique can be found in Figure 4.6.

For implementation, a regex-based approach was primarily used, as parsing PDF files is complex. In addition to regex queries, a function was implemented to locate the end of a dictionary, as regex alone cannot parse nested open–close structures.

First, the document is searched for a page object (i.e., an object with a `/Type/Page` dictionary entry). A valid font must be embedded here for the stream to use. To achieve this, it proved most effective to match the object syntax `(\d+ \d+ obj.*?endobj)` and check for a `/Type/Page` entry in either the header or footer of the object’s main dictionary. This prevents false positives caused by matching nested dictionaries.

The insertion of the font within the existing structure appears as follows:

```
Page Dictionary
  Resources Dictionary
    Font Dictionary
      /Font124573 << /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>
```

A challenge in implementation is that a dictionary, if it exists, can be embedded either directly (nested like `<< dict >>`) or indirectly (referencing an object containing that dictionary, e.g., `objnum 0 R`). Because the path must be traced from the Page Dictionary to the Resource Dictionary and then to the Font Dictionary, the implementation must account for the non-existence, direct embedding, and indirect reference of each dictionary. Finally, the custom font definition is added to the Font Dictionary for use in the stream object.

The stream object is created as a separate object at the end of the PDF body, incrementing the maximum object number. The stream structure is as follows:

```
BT /{fontname_str} 24 Tf 3 Tr 0 0 Td ({content.decode('ascii')}) Tj ET
```

Here, `BT` signals the beginning of text, followed by the font name and font size referred to by the `Tf` keyword. `3 Tr` sets the text render mode to invisible. `0 0 Td` defines the position on the page (bottom left). The payload follows within parentheses. `ET` marks the end of the text.

After creating the stream object, a reference to it is added by appending it to the Content array in the previously located page object. If the array does not exist, it is created. Ultimately, the page object contains a valid stream referencing a font correctly embedded within the page resources.

The primary challenge in implementing this technique is maintaining PDF validity and preventing corruption. Given the diversity of PDF structures, this remains a challenge not fully resolved by the proof-of-concept implementation. Linearized PDFs, in particular, lack proper support regarding the repair of the xref table and trailer. Although not all generated PDFs strictly conform to the standard, the resulting polyglots open without issues in multiple PDF viewers.

A significant limitation of this technique is the restriction to valid text characters; consequently, it is suited only for script polyglots.

4. Polyglot File Generation Techniques

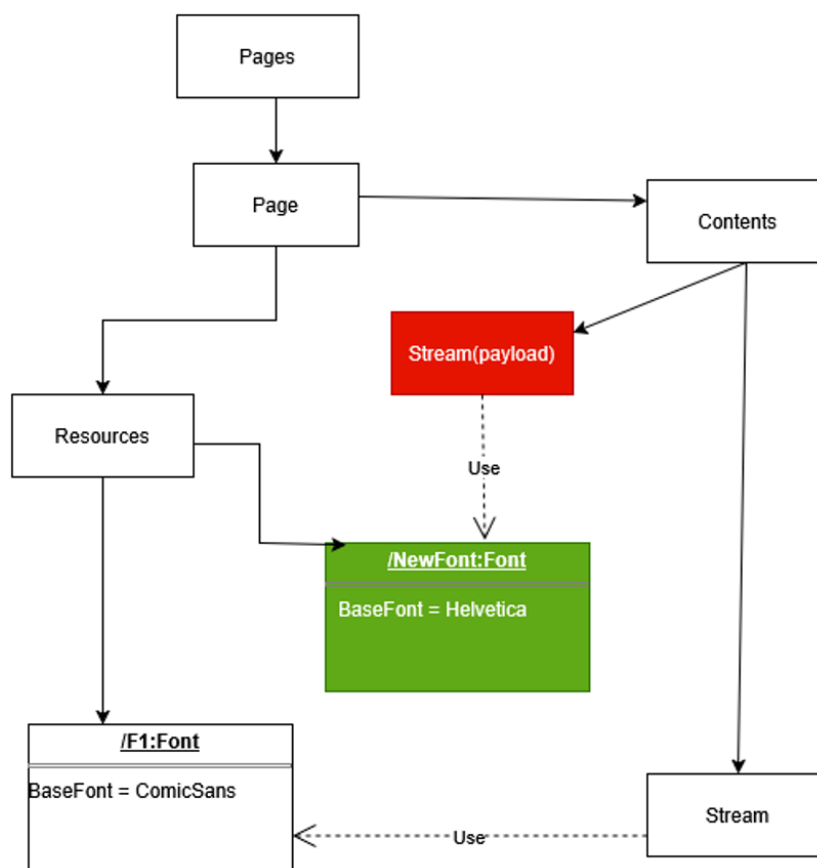


Figure 4.6.: Visualization of the structures injected into a simple PDF structure

A minor downside is that, because the text to print is enclosed in parentheses (e.g., `(text)`), parentheses inside the payload string must be balanced. However, since most script languages enforce balanced parentheses (outside of string definitions and comments), this is rarely a significant issue.

The major advantage of this technique is that the text is parsed and exists within the PDF (selectable), making it difficult to detect via structural analysis or parse trees.

4.7. Taxonomy of Presented Techniques

Based on the description in 2.5, techniques 4.3 and 4.4 fall under the category of Parasites, as they embed data in non-essential chunks. However, techniques 4.6, 4.1, 4.2, and 4.5 do not fit clearly into any single category. While they resemble parasites, they reside in essential regions of the file. The advantage of this placement is that they cannot be detected by mere structural analysis. For images, the covert files cannot be stripped away without corrupting the file. Similarly, stripping valid contents from PDFs risks false

4.7. *Taxonomy of Presented Techniques*

positives and compromises file integrity.

5. Implementation

5.1. Generation, detection and evaluation framework

To collect sample images, the tool Bulk Downloader for Reddit¹ was used to scrape the r/Pics subreddit. The acquired dataset consisted primarily of JPEG files. Consequently, FFmpeg was used to convert all the JPEG files to PNG and BMP images. The BMP images were downsampled to reduce evaluation time. The sample PDFs were taken from the SAFEDocs corpus² and filtered for a maximum size of 10MB to speed up the evaluation.

In order to evaluate both generation and detection tools, a framework for generation and detection was created using Python.

The generator iterates over the given techniques and samples to create polyglot files as well as metadata such as the types, source files, file hashes, and the used technique for later analysis. Traditional stack and parasite polyglots were generated using the Mitra tool [Alb20]. The Mitra tool proved highly effective for handling the employed cover formats, because even though it does not explicitly support them all, supplying the `-f` (force) argument allows the embedding of arbitrary data which suffices for versatile covert formats. The implemented architecture makes it easy to modify the generator classes (techniques) to run, which covert formats to implement, and the number of samples to use.

The detection program then runs all generated files against all configured detectors. It then normalizes the output by converting the differing type naming conventions of the detection tools to a single label (see [FIF⁺25] for more information on this issue). The program captures whether the file is found to be a polyglot (more than 2 matches exist), as well as the exact matches that were detected. Some false-positive prone matches were excluded as described below in 5.1.1. It is necessary to note that due to the use of the ‘signal’ library to implement timeouts, the detection loop will only work on Unix-based systems.

5.1.1. Detector tools

This section describes how the tools used for detection were invoked. Additional format detections that occurred for a majority of the sample host files were removed. This was deemed a realistic modification as a high false-positive rate is very unfavourable for the tools when deployed for a real task. These modifications are also specified below.

To enable the ‘file’ utility to detect polyglot files, we ran the command

¹<https://github.com/Serene-Arc/bulk-downloader-for-reddit>

²<https://digitalcorpora.org/corpora/file-corpora/cc-main-2021-31-pdf-untruncated/>

5.1. Generation, detection and evaluation framework

```
1 file --keep-going --mime-type path
```

and had to remove `application/octet-stream` and `text/plain` detections because they are generalized classifications that match every binary or text file.

For Magika, we used two private functions to get the internal predictions:

```
1 out, features = magika._get_result_or_features_from_path(path)
2 preds = magika._get_raw_predictions([(path, features)])
```

then the predictions were associated with their labels from `magika._model_config.target_labels_space`. A label is then added to the detected types if its confidence score is greater than 5%. It was not tested whether adjusting this threshold might increase performance.

Polydet was imported as a library and its scan method was invoked on the file to test. MP3 detections were removed as it detected it on most monoglots.

For Polyfile, the `MagicMatcher` class was imported and the `match` method invoked on the file's binary data. As it outputs the found matches, which do not necessarily have to include a type, all matches without a mime-type associated with them were discarded. Furthermore, the detection of `text/x-diff` was removed on BMP files because it was detected on every monoglot BMP.

6. Evaluation

6.1. Testing of polyglot file functionality

The functionality of the generated files was mainly tested using HTA and RAR covert files on different samples. Files were manually generated and tested for the functionality of both the covert and overt formats. PHP covert files were tested for functionality, but only in limited quantities.

6.2. Evaluation on detection tools

The evaluation dataset consists of 1,126 polyglot files generated using the described techniques, as well as 936 polyglot stack and parasite files generated using the Mitra tool. The same corpus of samples was used for both the presented techniques and Mitra. The 62 sample files used in the generation were also evaluated to calculate false-positive rates. The covert types used consisted of PHP, JavaScript, and PNG. While [KOS⁺25] found HTA to be one of the most important covert files used, it was decided against using it because Magika was not trained on those files. Due to the structural similarity, we chose to use JavaScript instead.

6.2.1. False positive rate

Having a low false-positive rate is very important because if a detection tool were to be deployed in a real environment, monoglot files would most likely be far more prevalent than polyglot files. This would lead to very poor precision even when the false-positive rate is seemingly low. PolyFile had the most false-positive detections, detecting false positives among both script and image files. Polydet only had false positives on image files. It was found through manual inspection that both PolyFile and Polydet incorrectly detected ZIP within a monoglot PNG file, probably due to a ZIP End of Central Directory magic number being present in the IDAT segment by chance. Magika detected false positives only on script files. One should also note that this evaluation is inherently limited by the low sample size of 62 monoglot files.

6.2.2. Polyglot file detection

The recall was calculated based on whether more than two file formats were detected. The ‘file’ tool did not detect any polyglot files. This might be due to the covert formats employed and the limited sample size. Magika achieved a recall of 17.7%. It detected 33.3% of Mitra polyglots but had a very low recall when detecting polyglots from the

6.2. Evaluation on detection tools

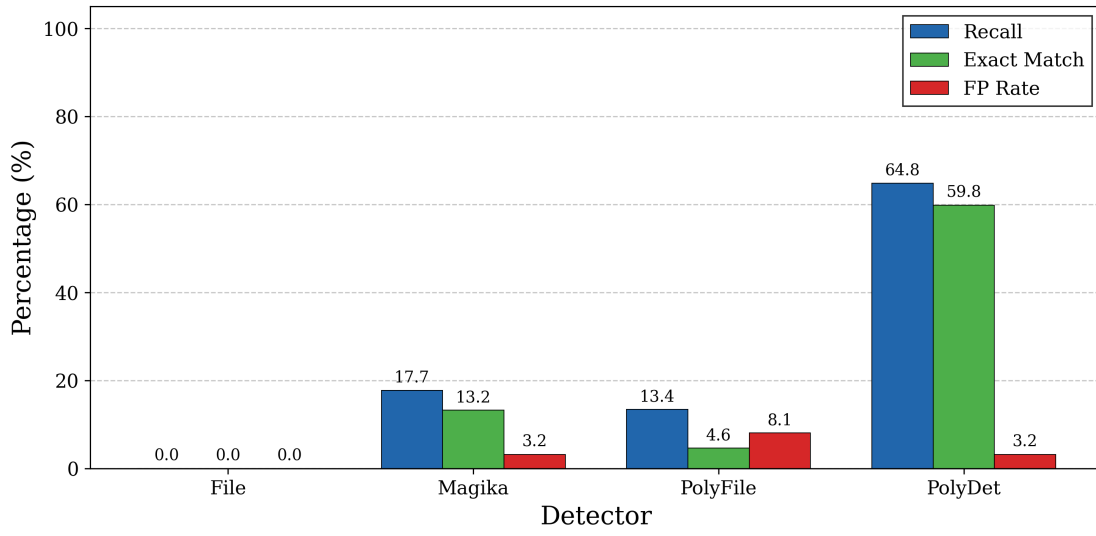


Figure 6.1.: Comparison of detection tools

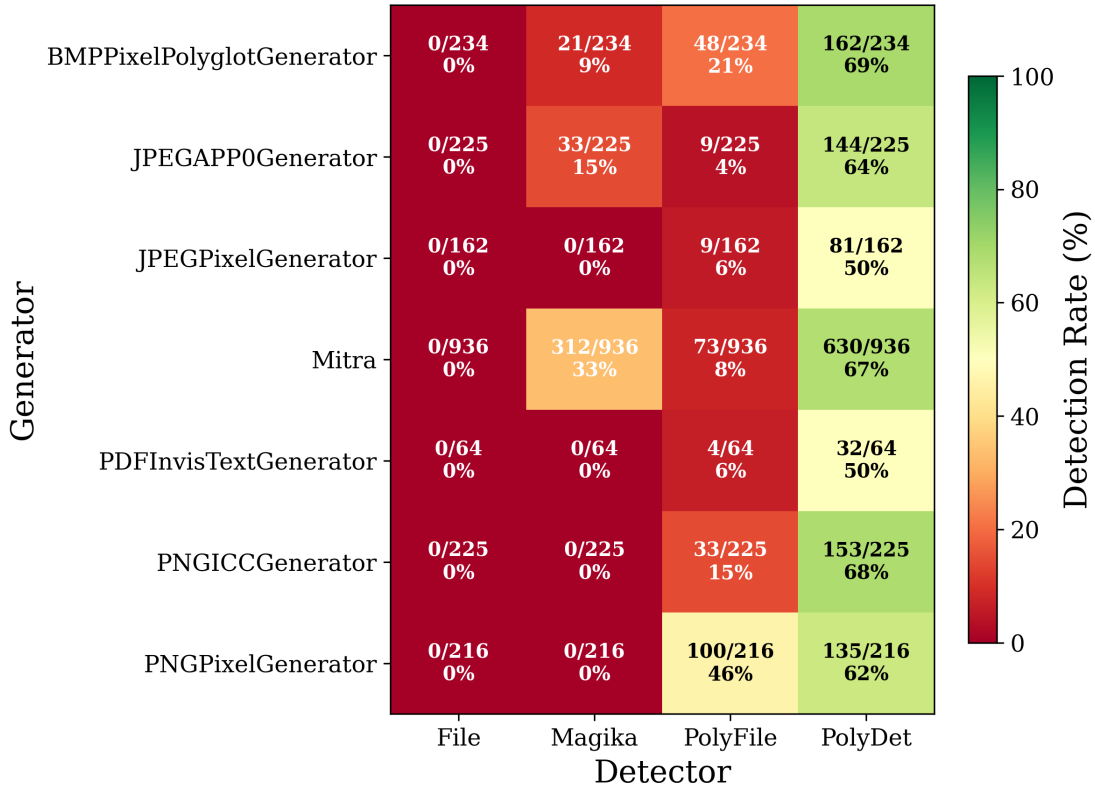


Figure 6.2.: Heatmap of polyglot detection rates

6. Evaluation

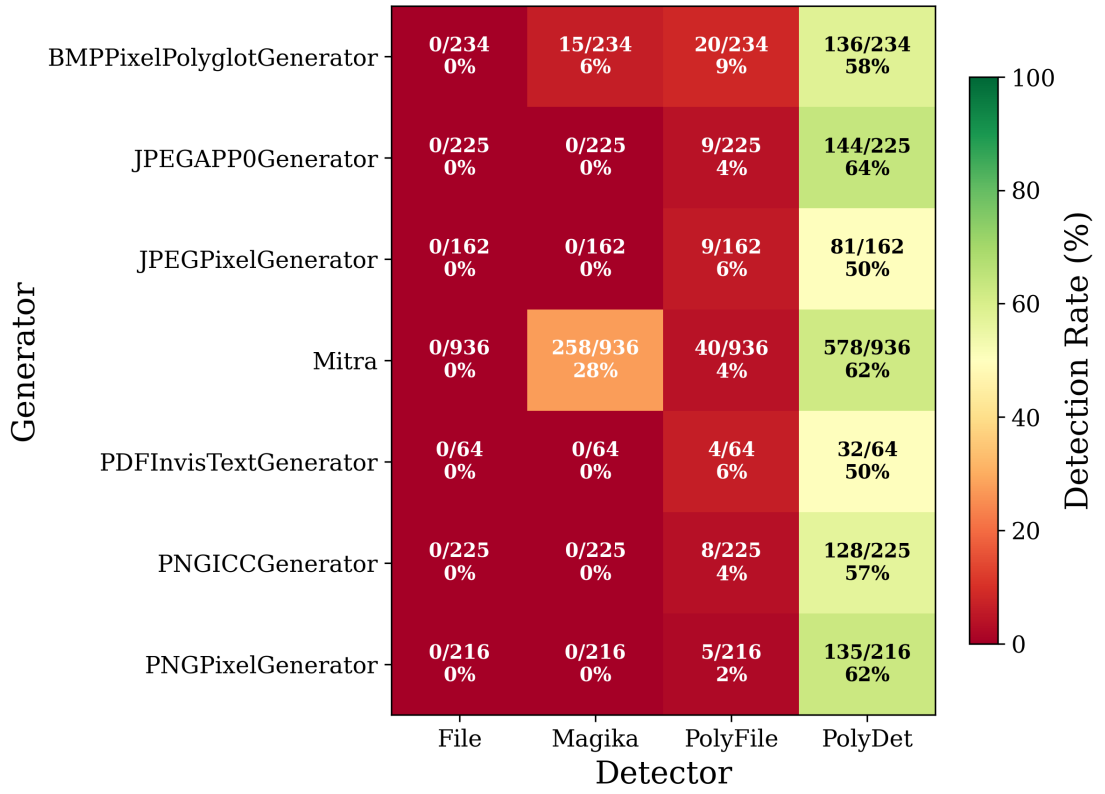


Figure 6.3.: Heatmap of exact file type detection rates

techniques we presented. The other tools did not differ significantly by technique. PolyFile performed unexpectedly poorly for a specialized detection tool with a recall of only 13.4%, greatly differing from the results found in [KOS⁺25]. It detected 50% of covert files injected into PNG pixel data but mostly detected them as type "text/x-diff" and not the exact script language. Polydet performed the best out of the detectors with a recall of 64.8%. It did not differ much between techniques. Polydet also managed to detect the exact format on a majority of files.

Limitations include the sample size of only 2,062 polyglot files tested. Furthermore, only three covert formats were employed. In order to be compatible with size restrictions on the techniques, most covert files were quite small, ranging from 1–3 Kilobytes. It is important to note that the 0% detection of Magika on the PNG ICC embedding technique is due to the ICC size surpassing the 1024 bytes from the beginning of the file, which Magika uses to create a feature vector.

6.2.3. Reproducibility

The generation and detection framework, along with links to the sample files used for evaluation, is available at <https://github.com/1337Name/Poly-Thesis>.

6.3. Evaluation on CDR tools

6.3.1. Tested tools

We tested generated polyglots against three CDR tools:

- **IMSan:** This is a simple tool presented by [KOS⁺25]. It uses the Pillow library to gather the most important data of an image; it then creates a new image and inserts this data, thus reconstructing and stripping all non-vital data contained in the original file. It managed to sanitize 100% of the files tested. As it was not released, we reconstructed the tool as per the description in the paper.
- **Glasswall Halo:** This is a commercial tool for file sanitization and repair available for testing at <https://halo.glasswall.com/clean-a-file>. It can sanitize and repair files as well as assign them a risk level of either low, medium, or high. One has to note that this instance uses the default policy, but custom policies and different sanitization products, like an SDK, are available¹, which might give different results.
- **OPSWAT Metadefender:** This tool is available for testing at <https://metadefender.com/>. It uses its DeepCDR solution to sanitize files. It can also use YARA scanning to find suspicious strings in a file and assigns a threat score. It is again necessary to note that a different configuration might give different results.

¹<https://www.glasswall.com/products>

6. Evaluation

6.3.2. Test methodology

Due to a size restriction on Halo of 5MB per file, we limited the test corpus to fit that restriction. Files were then selected from each generator, including at least one RAR (if the generator accepted binary data) and one script file. The files were then tested against each CDR tool.

6.3.3. Results

Format	Technique	IMSan	Halo	Metadefender
BMP	Pixel Injection	Not Sanitized	Not Sanitized	Sanitized
JPEG	APP0 Injection	Sanitized	Sanitized	Sanitized
JPEG	Scan Data Injection	Sanitized	Not Sanitized	Sanitized
PNG	ICC Profile Injection	Sanitized	Error	Sanitized
PNG	IDAT Pixel Injection	Sanitized	Not Sanitized	Sanitized
PDF	Stack (Append)	N/A	Sanitized	Sanitized
PDF	Mitra Stream	N/A	Sanitized	Not Sanitized
PDF	Invisible Text	N/A	Not Sanitized	Not Sanitized

Table 6.1.: CDR Sanitization Results by Polyglot Technique

The evaluation (see Table 6.1) shows that all tools managed to sanitize the parasitic techniques as expected. However, no tool managed to sanitize every one of the techniques presented here. The IMSan tool effectively removed all metadata and reconstructed the file. This reconstruction got rid of the IDAT and Scan data injections by re-encoding/recompressing the image data. OPSWAT Metadefender followed the same strategy.

IMSan fails to sanitize the BMP injection because the pixel data is already in standard encoding. It seems that OPSWAT Metadefender approaches this issue by matching the suspicious strings/archives and subtly changing the byte values so as to corrupt and neuter the payload without affecting the image. The rest of the image stays unaffected.

Glasswall Halo struggled with the Pixel and Scan data polyglots and failed to sanitize them. This comes as a surprise, as they had documented this technique in a white paper [Mor25a]. It was found that Halo rejects every PNG with an ICC profile that includes the junk tag. When the ICC file is added without the junk tag, it is accepted; however, the now-benign ICC profile is stripped completely. This behavior of just stripping the complete ICC profile was observed in all CDRs.

It is notable that Halo and Metadefender completely stripped the APP0 segment in all JPEG files. While it is commonly ignored by parsers [Mor25a], this makes the file

non-compliant to the JPEG standard which requires the APP0 segment right after the SOI [ITU92]. IMSan reconstructed a default APP0 segment with no thumbnail data.

For PDF, both Halo and Metadefender removed simple appended data as expected. We then tested a parasitic Stream technique used by the Mitra tool [Alb20]. It works by creating a new stream object and indirectly referencing it in the Catalog dictionary (the root of a PDF's object tree) under a custom key of `‘/Payload‘`. Glasswall successfully managed to remove the stream. Metadefender, while able to remove unused objects, was seemingly tricked by the reference and thus did not sanitize the newly added stream. Both tools failed to sanitize the invisible text stream, which was validly embedded into the page structure.

7. Discussion

The evaluation has shown that there is still significant work to do in combating the threat of polyglot files. None of the detectors demonstrated a sufficiently high detection rate, and while testing was limited, the false-positive rate also seemed unsatisfactory. Furthermore, none of the tested CDR tools were able to sanitize all the presented techniques. It can also be observed that the embedding of covert files in semantically relevant data introduces a fundamental tradeoff between the level of sanitization and the usability or integrity of the file.

7.1. Detection of polyglot files

As shown above, none of the tools tested achieved a high rate of detection approaching 90-100%. Furthermore, specialized tools like PolyFile and Polydet suffered from detecting false positives in benign sample image files. Coupled with a high volume of benign files passed through the CDR, as in most real-world use cases, this can lead to significant usability concerns or a volume of flagged files too large to analyze manually.

For the specialized tools, which mainly relied on signature matching, the technique used to embed the payload seemed to make little difference in the detection rate. Interestingly, Magika seemed to be better at detecting the traditional polyglots generated with Mitra, even more so when considering exact matches only. It would be important to see if such behavior also occurs with other neural network-based identifiers such as PolyConv, which contrary to Magika, was trained on polyglot samples, but only of stack and parasite kinds [KOS⁺25].

It is also important to note that Magika (version 1.0.1) is fundamentally limited in its ability to detect polyglot files by using just 1024 bytes from the beginning and end of the file to create its feature vectors, and as such could easily be bypassed by embedding outside of this region.

7.2. Content Disarmament and Reconstruction

It has been shown that current CDR tools manage to do exceptionally well on removing any non-vital data from image files. One downside to this behavior is that while stripping pure metadata probably has little impact on the usability of the file, stripping certain structures such as ICC will reduce the fidelity of the sanitized picture.

It has also been shown that sanitizing images with the payload contained within vital pixel or scan data warrants additional considerations. OPSWAT Metadefender's approach of both corrupting matched suspicious structures and re-encoding the data was successful

in sanitizing all tested images. But this approach depends on the matching algorithm being successful as well as being aware of specific indicators for the embedded formats. Furthermore, there already exists documentation on a real attack where the lossless PNG format was used to hide a malicious BMP-HTA polyglot [KOS⁺25]. Just re-encoding the PNG file would not defend against such an attack. The most secure way to sanitize files thus seems to be an anti-steganographic approach similar to [BDD23] and the ones demonstrated in [Mor25b]. The downsides to this approach are that it is computationally more intensive and can reduce the image quality.

Due to the complexity of the PDF file format [Dub23][Rim24], the approach of merely stripping non-essential objects may be very hard to implement accurately, as one would have to know all the potential spaces a parser will ignore, as well as any text that is not shown by being invisible or hidden behind other objects as shown above or in [Mor25a]. This can be seen by the example of Metadefender removing unused objects but failing to remove them when they are referenced by a dictionary key that is ignored by parsers. A quick fix for these specific approaches might be to deflate all streams in the PDF. The approach taken by [Dub23] seems to be the most comprehensive and secure approach by rebuilding the entire PDF and recursively sanitizing its components. But to date, this approach has not been refined to be compatible with all PDFs. Another approach would be to flatten the PDF by rasterizing the visible page contents.¹ Both these approaches come with considerable downsides. One of them is that it would invalidate all digital signatures [Dub23]. It might also not be clear how corrupted PDF structures should be handled as this might lead to different contents displayed before and after sanitization. The time it takes to sanitize large PDF documents might also be an issue.

7.3. Future Work

7.3.1. On detection

For a better and more accurate view on different tools' performances, the current testing would have to be extended to many more file format combinations, as well as a much larger testing corpus. It would be interesting to see if neural network-based approaches such as PolyConv currently are able to detect polyglots that are not just of stack or parasite kinds but rather embedded in the most vital contents of the file. Further, it is important to find where, how, and at what scale such detectors can be employed in real-world environments. For this, it is vital to analyze the impact of even a low rate of false positives in an environment where true polyglots constitute only a fraction of the files that are analyzed.

7.3.2. On Content Disarmament and Reconstruction

It has been shown that, to combat more sophisticated threats than mere embedding in metadata or comments, it is often necessary to alter the byte representation of the most

¹For example <https://www.adobe.com/acrobat/hub/how-to-flatten-a-pdf.html>

7. Discussion

vital data of the file. Most work treating such approaches is focused on preventing exploits or steganographically embedded payloads. Thus, these approaches are very aggressive and can cause issues with file integrity and usability. It is necessary to see how much these issues impact the adoption of the technology. It might be useful to find an approach which strikes a balance between usability and security specifically for the threat of polyglot files. For example, image polyglot file embeddings are much more frail than steganographic approaches (as detailed in [Mor25b]). A bigger problem consists in finding that balance in regards to complex document formats. In that regard, it would also be useful to study what degree of usability impairment users are willing to tolerate. For example, Glasswall noted in a blog article that users becoming aware of CDR might use their personal email account to receive work documents [Gla24], which then poses an even bigger issue than the original threat was.

8. Conclusion

The goal of this thesis was to test the file security landscape against known and new techniques for creating polyglot files. To this end, six different generators were implemented: two based on traditional parasite techniques and four that embed data in structurally essential regions of the host file. We then evaluated them and other Mitra-generated polyglots against polyglot detection tools as well as the file type detection tool Magika. To facilitate this evaluation, we built a modular generation and detection framework in Python. We then found that the tested tools failed to reliably detect the generated polyglots and exhibited a significant false-positive rate. We later showed that, while effective at stripping cavities, comments, and metadata, none of the tested modern CDR tools managed to sanitize all of the introduced generation techniques. Thus, we hypothesize that there is a fundamental tradeoff between the level of sanitization and the usability and integrity of a file.

In particular, we found that the tool Polydet achieved the best performance for file detection with a binary and exact detection rate of about 60%. We also found that PolyFile and Polydet are especially prone to misidentifying other file types in benign image files, which poses a problem when deployed at a large scale. One noticeable pattern was that Magika performed much better at detecting traditional Mitra-generated stack and parasite polyglots than the techniques presented here. We then showed that all CDR tools managed to remove the more traditionally embedded image polyglots. However, both IMSan and Glasswall Halo failed at sanitizing some image file types, while OPSWAT Metadefender managed to sanitize every image. When it comes to PDFs, both commercial tools left the presented injection technique unsanitized.

It should be noted that the evaluation presented here is inherently limited by sample size and the use of only three covert file formats. Furthermore, the implementation of the introduced techniques is not guaranteed to produce working polyglots on every file, as they were tested only with files encountered in the sample corpus.

Overall, it was highlighted that it does not suffice to assume that polyglots only reside in specific strippable regions or structures. Rather, one must treat every part of the file as a potential hiding spot for covert and malicious files. A more comprehensive evaluation, as well as research on finding the correct balance between sanitization and usability, is left for future work.

Bibliography

- [ADG⁺22] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. How to abuse and fix authenticated encryption without key commitment. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3291–3308, Boston, MA, August 2022. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/albertini>.
- [Alb15] Ange Albertini. Abusing file formats; or, corkami, the novella. *International Journal of Proof-of-Concept or Get The Fuck Out*, 0x07(7):18–41, March 2015. Available at: <https://github.com/angea/pocorgtfo>.
- [Alb20] Ange Albertini. Mitra: A polyglot file generator. <https://github.com/corkami/mitra>, 2020. Accessed: 2026-01-19.
- [Ali25] Sameed Ali. *Applications of Language-Theoretic Security Towards System Security*. PhD thesis, Dartmouth College, Hanover, NH, 2025. URL: <https://digitalcommons.dartmouth.edu/dissertations/402>.
- [Ana22] Prashant Anantharaman. *Protecting Systems From Exploits Using Language-Theoretic Security*. PhD thesis, Dartmouth College, Hanover, NH, 2022. Dartmouth College Ph.D. Dissertations. 80. URL: <https://digitalcommons.dartmouth.edu/dissertations/80>.
- [AS23] Sameed Ali and Sean W. Smith. A survey of parser differential anti-patterns. In *2023 IEEE Security and Privacy Workshops (SPW)*, pages 105–116, 2023. doi:10.1109/SPW59333.2023.00016.
- [BDD23] Eli Belkind, Ran Dubin, and Amit Dvir. Open image content disarm and reconstruction, 2023. URL: <https://arxiv.org/abs/2307.14057>, arXiv:2307.14057.
- [BSS23] Henrik Brodin, Evan Sultanik, and Marek Surovič. Blind spots: Automatically detecting ignored program inputs, 2023. URL: <https://arxiv.org/abs/2301.08700>, arXiv:2301.08700.
- [CIP19] CIPA. Exchangeable image file format for digital still cameras: Exif version 2.32. CIPA Standard DC-008-Translation-2019, May 2019. Released May 29, 2019.

- [CL18] Julien Campion and Hugo Laloge. Polydet: Polyglot detection tool. <https://github.com/Polydet/polydet>, 2018.
- [CNE20] Aviad Cohen, Nir Nissim, and Yuval Elovici. Maljpeg: Machine learning based solution for the detection of malicious jpeg images. *IEEE Access*, 8:19997–20011, 2020. doi:10.1109/ACCESS.2020.2969022.
- [CW24] Kuo-Chien Chou and Ran-Zan Wang. Dual-message qr codes. *Sensors*, 24(10), 2024. URL: <https://www.mdpi.com/1424-8220/24/10/3055>, doi:10.3390/s24103055.
- [Deu96] P. Deutsch. DEFLATE compressed data format specification version 1.3. RFC 1951, RFC Editor, May 1996. URL: <https://datatracker.ietf.org/doc/html/rfc1951>.
- [DG96] P. Deutsch and J-L. Gailly. ZLIB compressed data format specification version 3.3. RFC 1950, RFC Editor, May 1996. URL: <https://datatracker.ietf.org/doc/html/rfc1950>.
- [DKUW14] Adrian Dabrowski, Katharina Krombholz, Johanna Ullrich, and Edgar R. Weippl. Qr inception: Barcode-in-barcode attacks. In *Proceedings of the 4th ACM Workshop on Security and Privacy in Smartphones & Mobile Devices*, SPSM '14, page 3–10, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2666620.2666624.
- [Dub23] Ran Dubin. Content disarm and reconstruction of pdf files. *IEEE Access*, 11:38399–38416, 2023. URL: <https://api.semanticscholar.org/CorpusID:258203067>.
- [ELM16] Guillaume Endignoux, Olivier Levillain, and Jean-Yves Migeon. Caradoc: A pragmatic approach to PDF parsing and validation. In *IEEE Security and Privacy Workshops (SPW)*, pages 126–139. IEEE, 2016. URL: <http://spw16.langsec.org/papers/endignoux-caradoc.pdf>.
- [FIF⁺25] Yanick Fratantonio, Luca Invernizzi, Loua Farah, Kurt Thomas, Marina Zhang, Ange Albertini, Francois Galilee, Giancarlo Metitieri, Julien Cretin, Alexandre Petit-Bianco, David Tao, and Elie Bursztein. Magika: AI-Powered Content-Type Detection. In *Proceedings of the International Conference on Software Engineering (ICSE)*, April 2025.
- [Gla24] Glasswall. Debunking the myths: understanding CDR and cybersecurity, 2024. URL: <https://www.glasswall.com/blog/debunking-the-myths-understanding-cdr-and-cybersecurity>.
- [Ham92] Eric Hamilton. JPEG file interchange format, version 1.02. Technical report, C-Cube Microsystems, September 1992. URL: <https://www.w3.org/Graphics/JPEG/jfif3.pdf>.

Bibliography

- [ICC22] ICC. Image technology colour management – architecture, profile format, and data structure. Specification ICC.1:2022, International Color Consortium, 2022. Profile version 4.4.0.0.
- [ISO08] ISO. Document management – Portable Document Format – Part 1: PDF 1.7. ISO Standard 32000-1:2008, International Organization for Standardization, Geneva, Switzerland, 2008.
- [ITU92] ITU-T. Digital compression and coding of continuous-tone still images. ITU-T Recommendation T.81, International Telecommunication Union, September 1992. Approved 18 September 1992. Also published as ISO/IEC 10918-1:1994. URL: <https://www.w3.org/Graphics/JPEG/itu-t81.pdf>.
- [JLE20] Dong-Seob Jung, Sang-Joon Lee, and Ieck-Chae Euom. Imagedetox: Method for the neutralization of malicious code hidden in image files. *Symmetry*, 12(10), 2020. URL: <https://www.mdpi.com/2073-8994/12/10/1621>, doi:10.3390/sym12101621.
- [KJ24] David Klein and Martin Johns. Parse me, baby, one more time: Bypassing html sanitizer via parsing differentials. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 203–221, 2024. doi:10.1109/SP54263.2024.00177.
- [KOC⁺22] Luke Koch, Sean Oesch, Amul Chaulagain, Mary Adkisson, Samantha Erwin, and Brian Weber. Toward the detection of polyglot files. In *Proceedings of the 15th Workshop on Cyber Security Experimentation and Test*, CSET ’22, page 120–128, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3546096.3546106.
- [KOS⁺25] Luke Koch, Sean Oesch, Amir Sadovnik, Brian Weber, Amul Chaulagain, Matthew Dixon, Jared Dixon, Mike Huettel, Cory Watson, Jacob Hartman, and Richard Patulski. On the abuse and detection of polyglot files. In *Proceedings of the ACM on Web Conference 2025*, WWW ’25, page 4810–4822, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3696410.3714814.
- [Mica] Microsoft. Bitmap storage. URL: <https://learn.microsoft.com/en-us/windows/win32/gdi/bitmap-storage>.
- [Micb] Microsoft. BITMAPFILEHEADER structure. URL: <https://learn.microsoft.com/en-us/windows/win32/api/wingdi/ns-wingdi-bitmapfileheader>.
- [Micc] Microsoft. BITMAPINFOHEADER structure. URL: <https://learn.microsoft.com/en-us/windows/win32/api/wingdi/ns-wingdi-bitmapinfoheader>.
- [Midd] Microsoft. BITMAPV4HEADER structure. URL: <https://learn.microsoft.com/en-us/windows/win32/api/wingdi/ns-wingdi-bitmapv4header>.

- [Mice] Microsoft. BITMAPV5HEADER structure. URL: <https://learn.microsoft.com/en-us/windows/win32/api/wingdi/ns-wingdi-bitmapv5header>.
- [Mor25a] Connor Morley. Polyglot research – images and PDF. Research whitepaper, Glasswall, November 2025. Version 1.3. URL: <https://docs.glasswall.com/docs/polyglot-research-unmasking-images-pdf>.
- [Mor25b] Connor Morley. Smudging the invisible ink – image steganography disruption with a focus on cover image degradation minimisation. Research whitepaper, Glasswall, November 2025. Version 1.1. URL: <https://docs.glasswall.com/docs/steganography-smudging-the-invisible-ink>.
- [PNG03] PNG Development Group. PNG (portable network graphics) specification, version 1.2. Technical report, W3C, November 2003. W3C Recommendation. URL: <https://www.w3.org/TR/png-3>.
- [Rim24] Hannah Rimser. Creating digitally signed schizophrenic PDF files. Bachelor’s thesis, Universität Wien, Wien, Austria, 2024. Supervised by Univ.-Prof. Dipl.-Ing. Mag. Dr.techn. Edgar Weippl.
- [Rob20] Michael Robinson. Looking for non-compliant documents using error messages from multiple parsers, 2020. URL: <https://arxiv.org/abs/2012.10211>, arXiv:2012.10211.
- [SPB12] Len Sassaman, Meredith L. Patterson, and Sergey Bratus. A patch for postel’s robustness principle. *IEEE Security Privacy*, 10(2):87–91, 2012. doi:10.1109/MSP.2012.31.
- [Tra19] Trail of Bits. Polyfile: A utility to identify and map the semantic structure of files. <https://github.com/trailofbits/polyfile>, 2019.

A. Appendix

A.1. Usage of AI tools

AI was used to add minimal docstrings to all Modules/classes/functions in the project. It was also used to smoothen the thesis's text flow, readability and to correct grammatical mistakes. Concretely, a raw version of the sections was manually written and later refined in those areas with AI usage. It was specifically instructed to not change the semantic meaning. AI was also used to create some Bib-text entries where they did not exist. Then they were verified to avoid wrong and hallucinated facts (e.g. the dates, access date and authors). AI was also used to create quick and dirty scripts to debug issues in generation and detection. Lastly it was used to convert the graph generating python script from using the plotly to using the matplotlib library, with special focus on readability, as the plotly graphs were badly readable when inserted to the thesis.